德州扑克（Texas Hold'em）板子

本板子默认 C++17，`ll` 统一表示 `long long`。它处理标准德州扑克：每名玩家有 2 张底牌，公共牌最多 5 张，最终从至多 7 张牌中选最好的 5 张。核心评估不依赖浮点数，返回一个可直接比较的整数分值；7 张牌只枚举 21 个五张组合，常数很小。

1. 代码

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// r: 2..14(A), s: 0..3，花色顺序 S/H/C/D
struct C {
    int s, r;
    bool operator<(const C &o) const { return r != o.r ? r < o.r : s < o.s; }
    bool operator==(const C &o) const { return s == o.s && r == o.r; }
};

// 输入 "As", "TD", "10h"；输出内部牌。大小写均可。
C cv(string x) {
    C c{-1, -1};
    if (x.empty()) return c;
    int p = (int)x.size() - 1;
    char q = (char)toupper((unsigned char)x[p]);
    string u = "SHCD";
    c.s = (int)u.find(q);
    string t = x.substr(0, p);
    if (t == "A" || t == "a") c.r = 14;
    else if (t == "K" || t == "k") c.r = 13;
    else if (t == "Q" || t == "q") c.r = 12;
    else if (t == "J" || t == "j") c.r = 11;
    else if (t == "T" || t == "t" || t == "10") c.r = 10;
    else if (t.size() == 1 && t[0] >= '2' && t[0] <= '9') c.r = t[0] - '0';
    return c;
}
istream& operator>>(istream &in, C &c) { string x; in >> x; c = cv(x); return in; }

// 0 高牌，1 一对，2 两对，3 三条，4 顺子，5 同花，
// 6 葫芦，7 四条，8 同花顺。v 越大牌越大。
ll sc(int c, initializer_list<int> a) {
    ll z = c;
    int i = 0;
    for (int x : a) { z = z * 15 + x; ++i; }
    while (i++ < 5) z *= 15;
    return z;
}

// 评估恰好 5 张牌；参数数组不会被修改。
ll e5(const C *a) {
```

```
    int q[15] = {}, f = 1;
    for (int i = 0; i < 5; ++i) ++q[a[i].r];
    for (int i = 1; i < 5; ++i) f &= a[i].s == a[0].s;

    int st = 0;
    for (int h = 14; h >= 5; --h) {
        bool ok = 1;
        for (int x = 0; x < 5; ++x) ok &= q[h - x] > 0;
        if (ok) { st = h; break; }
    }
    if (!st && q[14] && q[2] && q[3] && q[4] && q[5]) st = 5;
    if (f && st) return sc(8, {st});

    int a4 = 0, a3 = 0;
    vector<int> p, o;
    for (int r = 14; r >= 2; --r) {
        if (q[r] == 4) a4 = r;
        if (q[r] == 3) a3 = r;
        if (q[r] == 2) p.push_back(r);
        if (q[r] == 1) o.push_back(r);
    }
    if (a4) {
        int k = 0; for (int r : o) k = max(k, r);
        return sc(7, {a4, k});
    }
    if (a3 && !p.empty()) return sc(6, {a3, p[0]});
    if (f) {
        vector<int> k;
        for (int r = 14; r >= 2; --r) if (q[r]) k.push_back(r);
        return sc(5, {k[0], k[1], k[2], k[3], k[4]});
    }
    if (st) return sc(4, {st});
    if (a3) {
        vector<int> k; for (int r : o) k.push_back(r);
        return sc(3, {a3, k[0], k[1]});
    }
    if (p.size() >= 2) return sc(2, {p[0], p[1], o[0]});
    if (p.size() == 1) return sc(1, {p[0], o[0], o[1], o[2]});
    return sc(0, {o[0], o[1], o[2], o[3], o[4]});
}

// 评估 5..7 张牌；枚举所有五张组合，返回最佳分值。
ll ev(const C *a, int n) {
    ll z = 0; C t[5];
    for (int i = 0; i < n; ++i) for (int j = i + 1; j < n; ++j)
        for (int k = j + 1; k < n; ++k) for (int l = k + 1; l < n; ++l)
            for (int m = l + 1; m < n; ++m) {
                t[0] = a[i]; t[1] = a[j]; t[2] = a[k]; t[3] = a[l]; t[4] = a[m];
                z = max(z, e5(t));
            }
    return z;
}
```

```cpp
ll ev(const vector<C> &a) { return ev(a.data(), (int)a.size()); }

// 比较两手 5..7 张牌：1 表示 a 赢，0 平局，-1 表示 b 赢。
int cmp(const vector<C> &a, const vector<C> &b) {
    ll x = ev(a), y = ev(b); return x > y ? 1 : x < y ? -1 : 0;
}

// 生成完整 52 张牌。
vector<C> dk() {
    vector<C> d; d.reserve(52);
    for (int s = 0; s < 4; ++s) for (int r = 2; r <= 14; ++r) d.push_back({s, r});
    return d;
}

// 检查牌数、范围和重复牌；合法返回 true。
bool valid(const vector<C> &a) {
    bool u[4][15] = {};
    for (auto c : a) {
        if (c.s < 0 || c.s >= 4 || c.r < 2 || c.r > 14 || u[c.s][c.r]) return 0;
        u[c.s][c.r] = 1;
    }
    return 1;
}

// 蒙特卡洛估计单挑胜率；me 必须 2 张，bd 为 0..5 张公共牌，it 为模拟次数。
// 返回 (胜 + 和/2) / it；公共牌未发完时自动补齐。
double eq(const vector<C> &me, const vector<C> &bd, int it = 200000, ll sd =
712367821) {
    if (me.size() != 2 || bd.size() > 5 || !valid(me) || !valid(bd)) return -1;
    vector<C> d, z = me; z.insert(z.end(), bd.begin(), bd.end());
    if (!valid(z)) return -1;
    bool u[4][15] = {};
    for (auto c : z) u[c.s][c.r] = 1;
    for (auto c : dk()) if (!u[c.s][c.r]) d.push_back(c);
    mt19937_64 g(sd); int w = 0, tie = 0, need = 5 - (int)bd.size();
    array<C, 7> a{}, b{};
    for (int itn = 0; itn < it; ++itn) {
        shuffle(d.begin(), d.end(), g);
        a[0] = me[0]; a[1] = me[1]; b[0] = d[0]; b[1] = d[1];
        for (int i = 0; i < (int)bd.size(); ++i) a[2 + i] = b[2 + i] = bd[i];
        for (int i = 0; i < need; ++i) a[2 + bd.size() + i] = b[2 + bd.size() + i] =
d[2 + i];
        ll x = ev(a.data(), 7), y = ev(b.data(), 7);
        if (x > y) ++w; else if (x == y) ++tie;
    }
    return (w + tie * .5) / it;
}

// 精确计算河牌已发完时的单挑胜率；me 必须 2 张，bd 必须 5 张。
double ex(const vector<C> &me, const vector<C> &bd) {
    if (me.size() != 2 || bd.size() != 5 || !valid(me) || !valid(bd)) return -1;
    vector<C> z = me; z.insert(z.end(), bd.begin(), bd.end());
```

```
    if (!valid(z)) return -1;
    bool u[4][15] = {};
    for (auto c : z) u[c.s][c.r] = 1;
    vector<C> d; for (auto c : dk()) if (!u[c.s][c.r]) d.push_back(c);
    array<C, 7> a{}, b{}; int w = 0, tie = 0, n = 0;
    for (int i = 0; i < (int)d.size(); ++i) for (int j = i + 1; j < (int)d.size();
++j) {
        a[0] = me[0]; a[1] = me[1]; b[0] = d[i]; b[1] = d[j];
        for (int k = 0; k < 5; ++k) a[k + 2] = b[k + 2] = bd[k];
        ll x = ev(a.data(), 7), y = ev(b.data(), 7);
        if (x > y) ++w; else if (x == y) ++tie; ++n;
    }
    return n ? (w + tie * .5) / n : -1;
}
```

2. 牌面和分值约定

- `C{s,r}` 中 `s=0,1,2,3` 依次为黑桃、红桃、梅花、方块，`r=2..14`，其中 `14` 是 A。
- `cv("As")`、`cv("10h")` 将字符串转成牌；也可以直接 `cin >> c`。
- `sc` 把牌型和最多五个破同点数压进一个整数。先比较牌型，再按从左到右的破同点数比较，因此 `ev` 的结果可以直接用 `<`、`==`、`>` 比较。
- A2345 顺子按 5 作为顺子最高点；同花顺的 A2345 也会正确处理。

牌型编号从小到大为：高牌、一对、两对、三条、顺子、同花、葫芦、四条、同花顺。皇家同花顺就是最高点为 14 的同花顺，不需要额外分支。

3. 常用调用

```
vector<C> a = {cv("As"), cv("Kd"), cv("Qc"), cv("Jh"), cv("Tc")};
ll x = ev(a);                          // 五张牌牌力

vector<C> h = {cv("As"), cv("Ah")};
vector<C> b = {cv("Ks"), cv("Qs"), cv("2d"), cv("7c"), cv("3h")};
ll y = ev(vector<C>{h[0], h[1], b[0], b[1], b[2], b[3], b[4]}); // 七张取最佳五张

int r = cmp(vector<C>{h[0], h[1], b[0], b[1], b[2], b[3], b[4]},
            vector<C>{cv("Kc"), cv("Kd"), b[0], b[1], b[2], b[3], b[4]});
double p = eq(h, {b[0], b[1]}, 1000000, 123456789); // 已知两张公共牌时估计胜率
double q = ex(h, {cv("Ks"), cv("Qs"), cv("2d"), cv("7c"), cv("3h")}); // 河牌精确胜率
```

`ev` 的第二个参数必须是 5、6 或 7；传入更少的牌没有定义。`cmp` 两边牌数可以不同，但都应至少 5 张。`valid` 只检查范围和重复，不检查德扑规则之外的输入。`eq` 是单挑（一个对手两张底牌），`it` 越大方差越小；固定 `sd` 可复现实验，传入不同种子可降低相关性。`ex` 只接受完整五张公共牌，逐一枚举对手底牌，结果是精确值。两个函数返回 `-1` 都表示参数非法。

4. 复杂度和适用范围

`e5` 为 `O(1)`，`ev` 最多评估 `C(7,5)=21` 个组合，时间和额外空间都是常数。`eq` 每次模拟洗牌整副剩余牌并评估两手七张牌，复杂度约为 `O(it * 52)`；适合赛场做胜率估计，不适合作为严格概率输出。

若公共牌已经完整，精确单挑概率只需枚举剩余两张对手牌（最多约一千种），可以把 `dk()` 后的组合循环替换为确定性枚举；若要精确枚举翻牌前所有发牌顺序，组合数会很大，建议使用本板子的 `eq` 或自行加剪枝。